want to behave differently for some MVC actions or Razor Pages, you should consider using a filter.

The middleware-versus-filters argument is a subtle one, and it doesn't matter which you choose as long as it works for you. You can even use middleware components inside the MVC filter pipeline, effectively turning a middleware component into a filter!

TIP The middleware-as-filters feature was introduced in ASP.NET Core 1.1 and is also available in later versions. The canonical use case is for localizing requests to multiple languages. I have a blog series on how to use the feature here: http://mng.bz/RXa0.

Filters can be a little abstract in isolation, so in the next section we'll look at some code and learn how to write a custom MVC filter in ASP.NET Core.

21.4 Creating a simple filter

In this section, I show you how to create your first filters; in section 21.5 you'll see how to apply them to MVC controllers and actions. We'll start small, creating filters that only write to the console, but in chapter 22 we look at some more practical examples and discuss some of their nuances.

You implement a filter for a given stage by implementing one of a pair of interfaces, one synchronous (sync) and one asynchronous (async):

- *Authorization filters*—`IAuthorizationFilter` or `IAsyncAuthorizationFilter`
- *Resource filters*—`IResourceFilter` or `IAsyncResourceFilter`